

Set 36: Binomial Hypothesis Testing

Skill 36.1: Distinguish between mean hypothesis test and binomial hypothesis tests

Skill 36.2: Identify the sample size and the number of successes in a dataset

Skill 36.3: Apply the `numpy.random.choice()` function to simulate a success-or-failure outcome

Skill 36.4: Simulate a null distribution

Skill 36.5: Calculate and interpret an interval of expected sample outcomes

Skill 36.6: Calculate and interpret a one-sided p-value

Skill 36.7: Calculate and interpret a two-sided p-value

Skill 36.8: Use the `binomtest()` function to calculate the p-value

Skill 36.1: Distinguish between mean hypothesis test and binomial hypothesis tests

Skill 36.1 Concepts

In the previous lesson, we used a one-sample t-test to investigate claims about a population mean. In this lesson, we will investigate binomial hypothesis testing, which is used when the variable of interest has only two possible outcomes, such as success/failure, yes/no, or show up/don't show up. Rather than comparing a sample mean to an expected population mean, a binomial test compares the number or proportion of successes in a sample to an expected population probability.

For example, suppose an airline expects 90% of ticketed passengers to show up for a flight. If only 80 out of 100 ticketed passengers actually show up, we can use a binomial test to investigate whether this result is significantly different from the expected 90% show-up rate.

Binomial tests and one-sample t-tests are both types of hypothesis tests, but they are used in different situations:

- *Binomial* tests are used for *binary categorical data* and compare a sample result to an expected probability or proportion.
- *One-sample t-tests* are used for *quantitative data* and compare a sample mean to an expected population mean.

Below summarizes these differences,

If the question is about...	Example	Use this test
an average	average fish weight, average temperature, average test score	One-sample t-test
a proportion of successes	proportion who show up, proportion who vote yes, proportion of made shots	Binomial test

In Python, a binomial test can be run with a single line of code. However, simply running the function is not enough. To choose the correct test, avoid mistakes, and interpret the results meaningfully, it is important to understand the reasoning behind the test. In this lesson, we will use simulation in Python to build that conceptual understanding.

Skill 36.1 Exercise 1

Skill 36.2: Identify the sample size and the number of successes in a dataset

Skill 36.2 Concepts

In the previous section, we learned that a binomial hypothesis test is appropriate when each observation has only two possible outcomes and we want to test a claim about a population proportion. Once we know that a binomial test is the correct choice, the next step is to identify the values we need from the data.

The dataset below represents a monthly report from a website called Live-it-LIVE.com - a website that sells all the necessary props and costumes to recreate iconic movie scenes at home.

	timestamp	purchase	item
0	2020-01-17 17:23:06	y	cue cards - love actually
1	2020-01-25 17:09:39	n	NaN
2	2020-01-25 05:22:01	n	NaN
3	2020-01-18 04:33:40	y	t-rex - jurassic park
4	2020-01-24 17:24:52	n	NaN
5	2020-01-15 05:05:12	n	NaN
6	2020-01-01 03:51:44	n	NaN
7	2020-01-15 03:19:54	n	NaN
8	2020-01-01 18:17:08	n	NaN
9	2020-01-29 17:09:57	n	NaN

Live-it-LIVE.com reports that about 10% of visitors usually make a purchase during this time of year. Because the checkout page had a bug this month, the company wants to know whether the true purchase rate was lower than expected. Before we can run a hypothesis test, we must first use the dataset to determine the sample size and the number of purchases.

In this example, each row of the dataset represents one website visitor, so the total number of rows gives us the number of visitors. A “success” in this context means that a visitor made a purchase, so we can count the number of rows where the purchase column is “y”.

Code

```
import pandas as pd
monthly_report = pd.read_csv("data.csv", delimiter="\t")
#calculate and print sample_size:
sample_size = len(monthly_report)
print(sample_size)
#calculate and print num_purchased:
num_purchased = len(monthly_report[monthly_report.purchase == "y"])
print(num_purchased)
```

Output

```
500
41
```

Skill 36.2 Exercise 1

Skill 36.3: Apply the `numpy.random.choice()` function to simulate a success-or-failure outcome

Skill 36.3 Concepts

In the last exercise, we found that 500 people visited Live-it-LIVE.com during the month, and 41 of them made a purchase. If each visitor truly had a 10% chance of making a purchase, we would expect about 50 purchases out of 500 visitors. Since 41 is lower than 50, we might wonder whether the true purchase probability was actually lower than 10% this month.

However, sample results do not usually match the expected value exactly. Even if the true purchase probability were still 10%, the observed number of purchases could be a little higher or lower than 50 just because of random chance. To better understand this idea, let's look at a more familiar probability example: flipping a fair coin. We can simulate a coin flip in Python using the `numpy.random.choice()` function:

Code
<pre>import numpy as np flip = np.random.choice(['heads', 'tails'], size=1, p=[0.5, 0.5]) print(flip)</pre>
Output
heads

If we run this code (or flip a real coin) a few times, we'll find that — just like we can't know ahead of time whether any single visitor to Live-it-LIVE.com will make a purchase — we can't predict the outcome of any individual coin flip.

If we flip a fair coin 10 times in a row, we expect about 5 of those coins to come up heads (50%). We can simulate this in python by changing the size parameter of `numpy.random.choice()`:

Code
<pre>import numpy as np flip = np.random.choice(['heads', 'tails'], size=10, p=[0.5, 0.5]) print(flip)</pre>
Output
['heads' 'heads' 'tails' 'tails' 'heads' 'heads' 'heads' 'tails' 'tails' 'tails']

It's perfectly reasonable that we could get only four heads, or maybe six or seven! Because this is a random process, we can't guarantee that exactly half of our coin flips will come up heads. Similarly, even if each Live-it-LIVE visitor has a 10% chance of making a purchase, that doesn't mean we expect exactly 10% to do so in any given sample.

Returning to our live-it-LIVE.com, we know that each visitor has a 10% chance of making a purchase. Below, illustrates how we can simulate one visitor.

Code
<pre>one_visitor = np.random.choice(['y', 'n'], size=1, p=[.1, 1 - .1]) print(one_visitor)</pre>
Output
['n']

Next, we can simulate 500 visitors – the number of monthly visitors to the site,

Code
<pre>one_visitor = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(one_visitor)</pre>
Output
<pre>['n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'y' 'n' ... 'n' 'n' 'n' 'n' 'n' 'n' 'n' 'y' 'n' 'n' 'n' 'n' 'n' 'n' 'y' 'n' 'n' 'n']</pre>

Skill 36.3 Exercise 1

Skill 36.4: Simulate a null distribution

Skill 36.4 Concepts

The first step in running a hypothesis test is to form a *null hypothesis*. For the question of whether the purchase rate at Live-it-LIVE.com was different from 10% this month, the null hypothesis describes a world in which the true probability of a visitor making a purchase was exactly 10%, but by random chance, we observed that only 41 visitors (8.2%) made a purchase.

In the code example below, we build on the code from the previous exercise by calculating the number of the simulated visitors who made a purchase. When we run the code several times, notice we get different outputs because there is a probability of getting values other than 50 or 10% of 500.

Code
<pre>simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(sum(simulated_monthly_visitors == "y")) simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(sum(simulated_monthly_visitors == "y")) simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(sum(simulated_monthly_visitors == "y")) simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(sum(simulated_monthly_visitors == "y")) simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(sum(simulated_monthly_visitors == "y")) simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[.1, 1 - .1]) print(sum(simulated_monthly_visitors == "y"))</pre>
Output
<pre>50 52 49 60 43</pre>

By running the same simulated experiment many times, we can get a sense for how much a particular outcome (like the number of purchases) varies by random chance. Consider the following code,

Code
<pre>null_outcomes = [] for s in range(10000): simulated_monthly_visitors = np.random.choice(['y', 'n'], size=500, p=[0.1, 0.9]) num_purchased = np.sum(simulated_monthly_visitors == 'y') null_outcomes.append(num_purchased) #calculate the minimum and maximum values in null_outcomes here: null_min = min(null_outcomes) null_max = max(null_outcomes) print(null_min) print(null_max)</pre>
Output
28 75

Note in the example above, we simulated the experiment 10000 times. This was an arbitrarily chosen large number because it's big enough that it will yield almost all possible outcomes for our experiment, and small enough that the simulation still runs quickly. From inspecting the output, we can see that the number of purchases varied between 27 and 80. While this is useful information, we can learn even more from inspecting the full distribution.

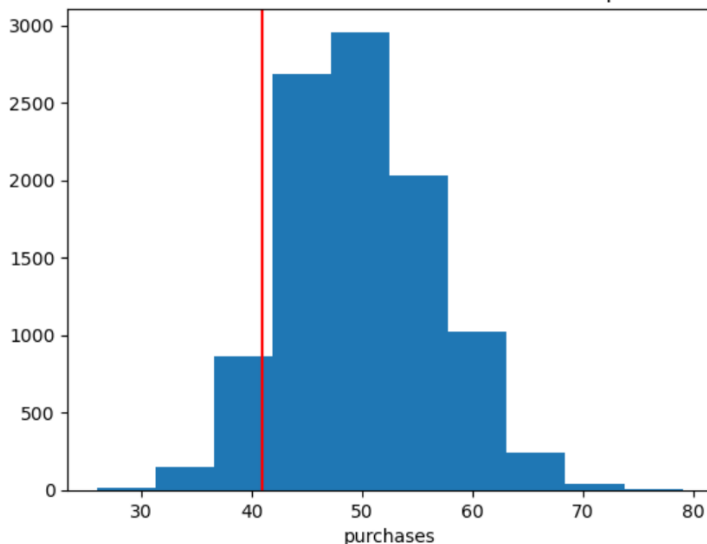
Below we plot a histogram of *null_outcomes* using *matplotlib.pyplot.hist()*. We can also add a vertical line at any x-value using *matplotlib.pyplot.axvline()*.

Code

```
import matplotlib.pyplot as plt
plt.hist(null_outcomes)
plt.axvline(41, color = 'r')
plt.xlabel("purchases")
plt.title("Distribution of Purchases in 10000 Simulations of sample_size = 500")
plt.show()
```

Output

Distribution of Purchases in 10000 Simulations of sample size = 500



We can see from the plot that the actual number of purchases is lower than expected, but not obviously rare enough from the graph alone to strongly reject the idea that the true purchase rate was still 10%.

Skill 36.4 Exercise 1

Skill 36.5: Calculate and interpret an interval of expected sample outcomes

Skill 36.5 Concepts

So far, we have looked at the full range of values in the null distribution. However, the smallest and largest simulated values happened only rarely. Instead of focusing on the entire range, it is often more useful to focus on the middle 95% of the distribution, which represents the outcomes we would expect to see most of the time if the null hypothesis were true.

In this example, the number of purchases in simulated samples of 500 visitors ranged from about 28 to 75, but those extreme values were uncommon. A 95% interval gives us a better sense of the typical results we would expect by random chance. For example, if each visitor truly has a 10% chance of making a purchase, then most random samples of 500 visitors would produce between 37 and 64 purchases.

We can calculate this middle 95% interval using the `np.percentile()` function by finding the 2.5th and 97.5th percentiles of the null distribution.

Code
<pre>confidence_int = np.percentile(null_outcomes, [2.5,97.5]) print(confidence_int)</pre>
Output
[37. 64.]

In the example above, we used the 2.5th and 97.5th percentiles to find the middle 95% of the null distribution. That means only 5% of simulated results fall outside this interval, with 2.5% in each tail.

We can use this interval to decide whether an observed result looks unusual under the null hypothesis. If the observed value falls outside the interval, it would be considered unusual and would give evidence against the null hypothesis. If it falls inside the interval, it would be considered fairly typical and could have happened by random chance.

In this case, the observed value of 41 falls inside the interval from 37 to 63. This suggests that 41 is a believable result if the true purchase rate were 10%, so the data do not provide strong evidence against the null hypothesis.

Skill 36.5 Exercise 1

Skill 36.6: Calculate and interpret a one-sided p-value

Skill 36.6 Concepts

P-value calculations and interpretations depend on the alternative hypothesis because the alternative describes the kind of difference from the null expectation that we are investigating.

For example, let's continue with our monthly website purchase example. Suppose we want to test the following hypotheses:

H₀: 10% of site visitors make a purchase

H_a: Less than 10% of site visitors make a purchase

This hypothesis test asks,

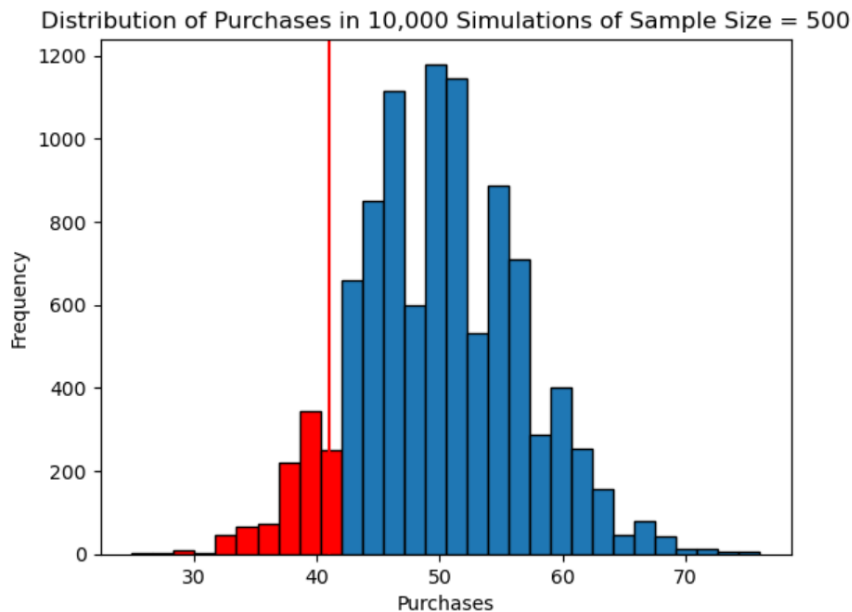
If the true purchase probability is 10%, what is the probability of observing 41 or fewer purchases in a sample of 500 visitors?

Earlier, we used a for loop to simulate 500 visitors 10,000 times under the null hypothesis and stored the number of purchases from each simulation in a list called *null_outcomes*.

To estimate the p-value, we calculate the proportion of simulated samples in which the number of purchases was 41 or fewer,

Code
<pre>values = sum(x <= 41 for x in null_outcomes) p_value = values / len(null_outcomes) print(p_value)</pre>
Output
0.1019

This calculation is equivalent to calculating the proportion of the histogram that is colored in red,



We estimated that the probability of observing 41 or fewer purchases is about 0.1019, or 10.19%, if the true purchase rate is really 10%. This probability is called a one-sided p-value.

Because the p-value is not very small, the observed result of 41 purchases is not especially unusual under the null hypothesis. That means the lower-than-expected number of purchases could reasonably have occurred by random chance alone.

Skill 36.6 Exercise 1

Skill 36.7: Calculate and interpret a two-sided p-value

Skill 36.7 Concepts

In the previous section, we calculated a one-sided p-value, which measured the probability of getting a result in one particular direction under the null hypothesis. In this section, we will estimate a two-sided p-value, which is used when we want to know whether the observed result is simply different from the expected value, regardless of direction.

In our website purchase example, the null hypothesis says that 10% of visitors make a purchase, so with 500 visitors we would expect about 50 purchases. In the previous section, we calculated the probability of observing 41 or fewer purchases, since we were interested only in whether the purchase rate was lower than expected.

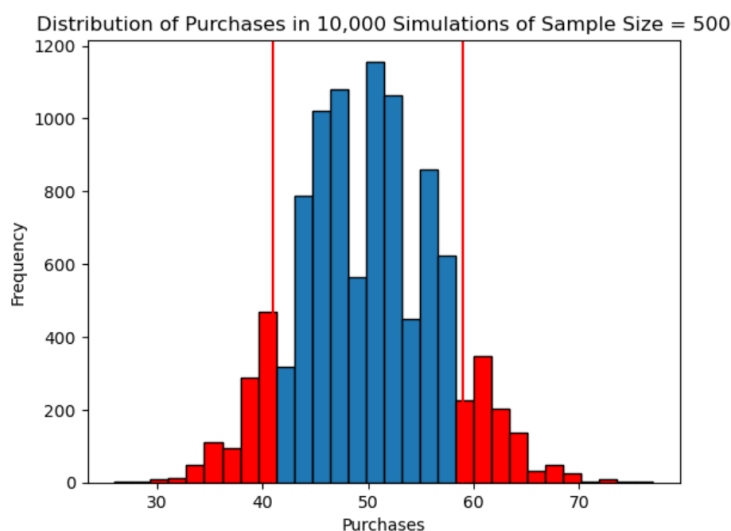
A two-sided test asks a broader question,

If the true purchase rate is 10%, what is the probability of observing a result at least as far from 50 as our observed value of 41, in either direction?

Since 41 is 9 purchases below the expected value of 50, we also include values that are 9 or more above 50. That means the two-sided p-value includes simulated results of:

- 41 or fewer purchases
- 59 or more purchases

So, the two-sided p-value is the proportion of the null distribution that falls in both red tails of the histogram.



To estimate the p-value, we calculate the proportion of simulated samples in which the number of purchases was 41 or fewer or 59 or more,

Code
<pre>values = sum((x <= 41) or (x >= 59) for x in null_outcomes) p_value_two_sided = values / len(null_outcomes) print(p_value_two_sided)</pre>
Output
0.2001

Because the p-value is not very small ($p=0.2001$), the observed result of 41 purchases is not especially unusual under the null hypothesis. In other words, a result this far from the expected value of 50, in either direction, could reasonably have occurred by random chance alone.

Skill 36.7 Exercise 1

Skill 36.8: Use the *binomtest()* function to calculate the p-value

Skill 36.8 Concepts

So far, we have conducted a simulated binomial hypothesis test for Live-it-LIVE.com step by step. In this section, we will combine that earlier code into a reusable function. Our function will still use simulation, so it will estimate the same p-value we would get from a formal binomial test, although the exact value may vary slightly from run to run because it is based on random simulation. Python's *binomtest()* function performs this test directly by using the number of successes, the number of trials, the hypothesized probability of success, and the chosen alternative hypothesis.

In the example below, we combine our simulation and p-value calculation into a single function called *simulation_binomial_test()*. This function estimates a one-sided left-tailed binomial test for any observed number of successes, sample size, and null probability of success.

Code

```
def simulation_binomial_test(observed_successes, n, p):
    #initialize null_outcomes
    null_outcomes = []

    #generate the simulated null distribution
    for i in range(10000):
        simulated_monthly_visitors = np.random.choice(['y', 'n'], size=n, p=[p, 1-p])
        num_purchased = np.sum(simulated_monthly_visitors == 'y')
        null_outcomes.append(num_purchased)

    #calculate a 1-sided p-value
    null_outcomes = np.array(null_outcomes)
    p_value = np.sum(null_outcomes <= observed_successes)/len(null_outcomes)

    #return the p-value
    return p_value

p_value = simulation_binomial_test(41, 500, .10)
print(p_value)
```

Output

0.1000

Because our *simulation_binomial_test()* function uses random simulation, it only estimates the p-value, so its output may change slightly each time it is run. In addition, it does not handle right-tailed or two-sided tests. SciPy's *binomtest()* function is more versatile because it can calculate the exact p-value for all three types of alternative hypotheses.

The *binomtest()* takes the number of observed successes, the total number of trials, and the hypothesized probability of success as inputs, and its default alternative is 'two-sided'. This can be changed with the alternative parameter. For example, *alternative='less'* runs a one-sided left-tailed test. In our example, we observed 41 purchases out of 500 visitors, and the null purchase probability is 0.10. The function call would look like this,

Code
<pre>from scipy.stats import binomtest p_value2 = binomtest(41, n = 500, p=0.1, alternative = "less") print(p_value2.pvalue)</pre>
Output
0.1000

Skill 36.8 Exercise 1